João Dionísio

March 27, 2026

Let's SCIP it!

APDIO Workshop: Optimization Software & Practice

What is SCIP?

SCIP — *Solving Constraint Integer Programs*

- Fastest non-commercial solver for mixed-integer programs (MIP/MINLP)
- Also handles satisfiability and constraint integer programs
- Supports exact arithmetic for MILPs
- Open-source under Apache 2.0 license
- Under active development for over 20 years (mostly at Zuse Institute Berlin)
- SCIP alumni work on Gurobi, Xpress, COPT, Hexaly, ...

Interfaces: C/C++, Python, Julia, Rust, Java, MATLAB

File formats: MPS, LP, CIP, ZIMPL, and more

Why do research on optimization?

Modern solvers are impressive, but they have limits:

- **Generic vs. specific:** solvers use general-purpose techniques. Exploiting problem structure can be orders of magnitude faster
- **Modeling power:** some formulations only become tractable with custom decomposition (e.g., Dantzig-Wolfe, Benders)
- **Beyond MIP:** many real-world problems have side constraints, nonlinearities, or combinatorial substructure that generic solvers handle poorly
- **Integration:** embedding domain-specific heuristics, cuts, or branching rules inside the solver beats solving + post-processing

Plugin-Based Architecture

SCIP's real strength: a **modular plugin system** that lets you customize every aspect of the solving process.

Core framework:

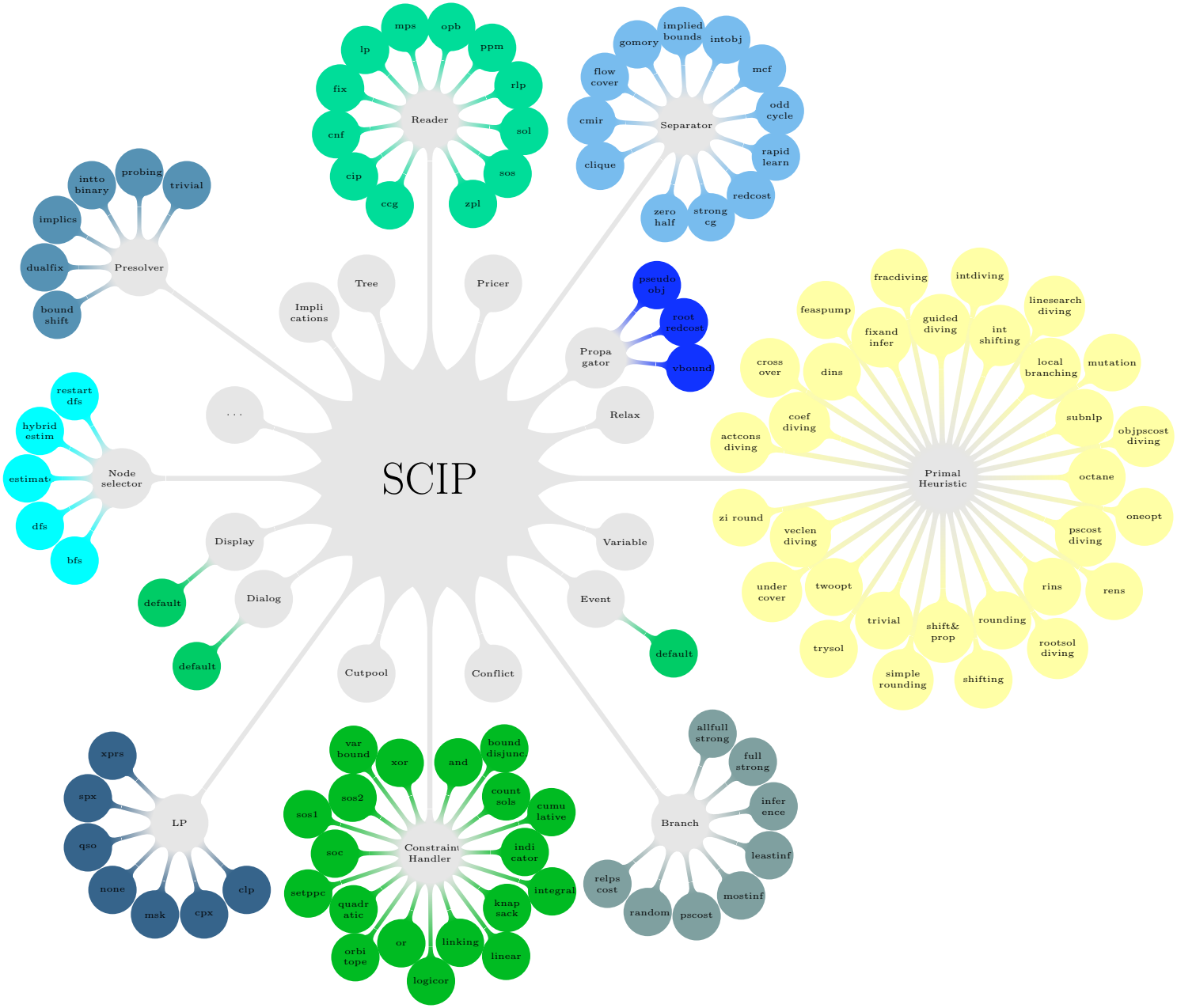
- Branch-and-bound tree
- LP solver (SoPlex)
- Solution & cut pools
- Conflict analysis
- Clique table

Plugins are modular:

- Each type has a defined interface
- Called by priority order
- Multiple plugins per type
- Extend in C, C++, or Python

$\Rightarrow$ Not just a solver — a **framework** for building solvers

Structure of SCIP



Plugin Types

Plugin	Purpose	When
`constraint handler`	feasibility, cuts, propagation	throughout
`separator`	cutting planes	after LP solve
`pricer`	column generation	after LP solve
`heuristic`	find feasible solutions	configurable
`branching rule`	split subproblems	at fractional node
`propagator`	tighten variable bounds	at each node
`presolver`	simplify before solving	before B&B
`relaxator`	custom relaxation bounds	at each node
`benders`	Benders decomposition	throughout
`event handler`	react to solving events	on trigger
`node selector`	pick next subproblem	after node solved

Plugin Types

Plugin	Purpose	When
`constraint handler`	**feasibility, cuts, propagation**	**throughout**
`separator`	**cutting planes**	**after LP solve**
`pricer`	**column generation**	**after LP solve**
`heuristic`	find feasible solutions	configurable
`branching rule`	**split subproblems**	**at fractional node**
`propagator`	tighten variable bounds	at each node
`presolver`	simplify before solving	before B&B
`relaxator`	custom relaxation bounds	at each node
`benders`	Benders decomposition	throughout
`event handler`	react to solving events	on trigger
`node selector`	pick next subproblem	after node solved

We will implement these in Parts 2 & 3!

Why SCIP?

As a black-box solver

- State-of-the-art MIP performance
- Nonlinear support (MINLP)
- 2000+ tunable parameters
- Rich presolving & heuristics
- Reads standard formats

As a framework

- Branch-and-price (pricer)
- Row generation (constraint handler)
- Custom branching rules
- Benders decomposition
- Event-driven callbacks
- Detailed statistics to verify your changes have an effect

You implement the problem-specific logic; SCIP handles the rest.

SCIP is both a solver and a framework for building custom solvers.

Today's Workshop

1. **Part 1 — Modeling:** build and solve MINLPs in PySCIPOpt
2. **Part 2 — Row Generation:** implement a constraint handler for TSP
3. **Part 3 — Branch-and-Price:** implement a pricer and branching rule for bin packing

Let's SCIP it!